TACS 2627 Assignment

QMDE — Quantum Model-Driven Engineering

DRAFT v0.7 (16/10/2026). Highlighted text marks decisions still to be confirmed.

1. Introduction 2
2. Overview of the Transformation Chain 2
3. Classes of Problems 3
4. Technologies 4
5. Part 0: Preparation 4
6. Part I 4
    1. Modelling Domain Analysis 5
    2. Modelling Language Design 5
    3. Deliverables 5
7. Part II 5
    1. Model-to-Model Transformations 5
    2. Model-to-Text Transformation 6
    3. Validation 6
    4. Deliverables 6
8. Use of LLMs 6
9. Evaluation 7
10. Final Remarks 7
11. References 7
12. Annex A: Classes of Problems 8
    Overview 8
    Intermediate Models 8
    A. Optimisation with QAOA 9
    B. Search with Logical Constraints (Grover) 10
    C. Search with Linear Arithmetic Constraints (Grover + reversible arithmetic) 11
    D. Optimisation with Grover Adaptive Search 12
13. Annex B: A Small End-to-End Example 14

1. Introduction

Quantum computers promise advantages for some classes of problems, such as combinatorial optimization, simulation of physical systems, search, sampling and certain machine-learning tasks. However, programming them today means working almost directly at the level of circuits: qubits, gates and measurements. This is the equivalent of writing assembly, and it creates a large gap between the people who have the problems (logisticians, financial analysts, physicists, data scientists) and the people who can write quantum programs.

Model-Driven Engineering (MDE) is well-suited to closing this gap. A domain expert should be able to state *what* the problem is in a domain-specific language (DSL), and a chain of model transformations, grounded in the published formulations for that domain, should derive *how* to solve it on a quantum computer, ending in executable code.

The goal of this assignment is to design and implement such a chain for one class of problems: a DSL (metamodel) for formulating problems of that class, a DSL (metamodel) for the solution, model-to-model (M2M) transformations between them, and model-to-text (M2T) transformations that generate Python code for Qiskit. You can use Large Language Models (LLMs) as assistants throughout the project, and to reflect critically on that use (see the section Use of LLMs).

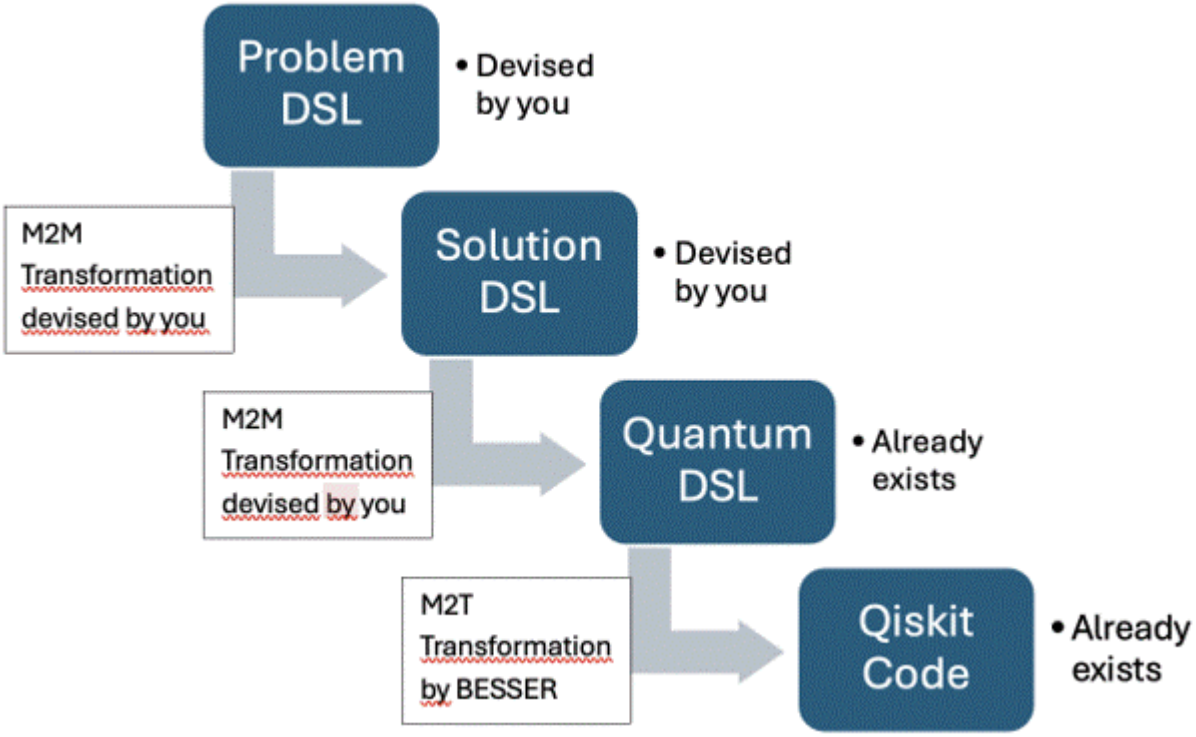
2. Overview of the Transformation Chain

Every group builds the same kind of chain, but for a different problem:

Layer	Artefact	Produced by
1. Problem DSL	Abstract syntax (metamodel + OCL) for describing problem instances of your class of problems	You (Part I)
2. Solution DSL	A mathematical formulation of the problem, defined per class in Annex A: QUBO, Boolean formula (CNF), linear integer constraints, or integer linear programme	You (Part I) M2M transformation T1 from Problem DSL (Part II)
3. Quantum DSL	An instance of your quantum circuit metamodel: registers, gates, parameters, measurements, repeated layers	Existent M2M transformation T2 from Solution DSL (Part II)
4. Code	Runnable Qiskit Python	Existent M2T transformation T3 from Quantum DSL (by BESSER)
5. Results	Execution on a simulator, compared against a classical baseline	Generated code + your validation scripts

Splitting the chain into two M2M steps keeps each transformation small, testable and traceable to the literature: T1 follows the domain formulation (e.g. how a scheduling problem becomes a Quadratic Unconstrained Binary Optimization (QUBO)), and T2 follows the algorithm construction (e.g. how a QUBO becomes a Quantum Approximate Optimization Algorithm (QAOA) circuit).

Some choices are not part of the problem itself but of how it is solved, for example, the QAOA depth, penalty weights, the encoding of integer variables, or the number of Grover rounds. These must be modelled explicitly, either in the solution DSL or in a small separate **solver configuration** model/DSL that is an extra input to the transformations. They must not be hard-coded in the transformations.



3. Classes of Problems

Each group works on **one problem from one of the four classes** below. The classes are built around NP-complete problems and their optimisation variants, which is where the quantum algorithms of the course apply, and they lead to different quantum algorithm families, so the M2M transformations across classes differ in substance, not only in domain. Each group submits its composition by October 5 (see Part 0); a class and a problem from its pool are then assigned to each group by the teachers by October 7, with about five groups per class. Annex A describes each class: the intermediate model (defined mathematically), the pool of problems, the minimum scope, stretch goals and starting references.

#	Class	Intermediate model (solution DSL)	Quantum algorithm
A	Optimisation with QAOA (graph problems, selection, assignment)	QUBO	QAOA
B	Search with logical constraints (configuration, puzzles, colouring)	Boolean formula (CNF)	Grover search
C	Search with linear arithmetic constraints (subset sum, partition, sum puzzles)	Linear integer constraints	Grover search with reversible arithmetic (adders, comparators)
D	Optimisation with linear objective and constraints (knapsack, set cover)	Integer linear programme (ILP)	Grover Adaptive Search

4. Technologies

Each group chooses **one** of the following technology stacks and uses it consistently.

Concern	EMF stack	BESSER stack [4]
Metamodels	Ecore	B-UML (BESSER metamodels)
(Object) Model	Ecore instance model	Object Model
Constraints	OCL (Eclipse OCL)	OCL (BESSER OCL support)
M2M	ATL, QVTo or Epsilon ETL	Python transformations over BESSER models

BESSER includes a quantum circuit metamodel [11] (registers, gates with control and anti-control qubits, measurements, sub-circuits), a graphical circuit editor in the BESSER Web Modeling Editor, and a Qiskit code generator with the Qiskit Aer simulator.

If necessary, a quantum circuit metamodel for **EMF** will be provided.

If you use BESSER for the development of the assignment, you must build using our own fork available at https://github.com/jacomecunha/BESSER, as updates may be necessary during the semester.

5. Part 0: Preparation

No previous knowledge of quantum computing is assumed. You are not expected to master quantum mechanics; you are expected to implement published constructions faithfully and to demonstrate, through validation, that you did. We will provide initial information about the topic:

1. **Introductory talk**, before the choice of classes. It covers only what the assignment needs: qubits and amplitudes, gates as matrices, measurement as sampling, parameterised circuits and the hybrid quantum–classical optimisation loop.
2. **Hands-on lab.** A lab class with introductory exercises.
3. **Reference chain.** A reference implementation of the chain for the example in Annex B (in BESSER).
4. **Reading pack.** For self-study, we recommend the free IBM Quantum Learning courses [10], in particular “Basics of Quantum Information” by John Watrous.

Groups. You should start forming groups of **3 students**. Each group should create a GitHub (or similar) repository and use it for development. The contribution of each student will be considered through their commits (number and content). Invite all the teachers for the repository (@jacomecunha, @joaopascoalfariafeup, @lessthelonely). You should **add the group to the following spreadsheet (student number and name, and the repository link) by October 5**: Groups

6. Part I

In this first part, you will study your problem and its class and design the two languages of the chain: the problem DSL and the solution DSL. We expect each group to come up with its own view of the domain; this will be explicitly evaluated. Annex A is a starting point, not a specification; the only fixed element is the mathematical meaning of the intermediate model of your class.

To achieve this goal, you must follow the steps below [1].

6.1. Modelling Domain Analysis

Start from the references for your class (Annex A) and find further examples: papers, tutorials, benchmark libraries and existing tools. Collect at least three concrete problem instances of different kinds and sizes; these will become your test cases. From this analysis, build two lists of concepts: one for each language (problem and solution). For each concept, define its attributes (e.g. weight, capacity, rotation angle) and its connections to other concepts (e.g. an edge connects two nodes; a gate acts on one or more qubits).

This list of concepts is a deliverable. We suggest a table similar to the one in Figure 7.4 of [1]. Also record, for each concept of the problem DSL, **which element of the solution DSL model it maps to and according to which reference**. This is the seed of transformation T1.

6.2. Modelling Language Design

Using the concept lists, define the two metamodels with your chosen stack. Not all constraints can be expressed in the metamodel, so use OCL to bring your metamodels closer to reality. Examples: every edge refers to existing nodes; capacities are positive; qubit indices are within the register size; the control and target qubits of a controlled gate are distinct; every parameter used in a gate is declared. The solution DSL must represent the intermediate model of your class as defined mathematically in Annex A; how you structure it is your design decision.

6.3. Deliverables

You should produce at least the following deliverables for Part I:

- The two lists of concepts for the two languages (problem DSL and solution DSL), including attributes, connections and the mapping to references. (can be part of the report)
- The metamodel of the problem DSL, with OCL constraints. (B-UML and an image)
- The metamodel of the solution DSL, with OCL constraints. (B-UML and an image)
- Example object models for one small problem instance, built by hand: the problem object model and the corresponding solution object model.
- A very short report (about 2 pages, **hand-written, not generated** ;) about what inspired your metamodels (e.g., which references you used) and the justifications you find necessary, so we can better understand your options. The report can be a markdown file in the repository.
- The LLM usage log for this part (see Use of LLMs).

The deadline for the first version of these deliverables is **October 21st (we will consider the last commit of this day)**. You may change them in Part II, but it is important to have a solid base in the following weeks.

7. Part II

In this second part, you complete the chain so that a problem written in your DSL is turned into executable quantum code, runs on a simulator, and produces an answer expressed in domain terms.

7. 1. Model-to-Model Transformations

Implement transformation **T1** (problem model → solution model) and transformation **T2** (solution model → quantum model). Each transformation rule must be traceable to the reference it implements: cite it in a comment next to the rule and in the report. Take the solver configuration into account (depth, penalty weights, encodings, number of iterations or steps). Any logic needed to solve the problem, such as decoding measured bitstrings back into a schedule, a colouring or a portfolio, must also be derived from the models and not added by hand afterwards.

7.2. Model-to-Text Transformation

Using the BESSER generator, from the quantum model (and, where needed, the problem and configuration models), generate:

5. **Qiskit Python**: a runnable script that builds the circuit, runs it on the Aer simulator, runs the classical optimisation loop for variational algorithms, and prints the result in domain terms.

Generated code must not be edited by hand. If something is wrong, fix the models or the transformations.

7.3. Validation

Show that your chain is correct. For each of your test instances (at least three, small enough to simulate, roughly up to 20 qubits), compare the quantum result against a classical baseline: brute force, or a classical solver chosen and found by you (e.g. dynamic programming, a SAT or ILP solver). Report a metric suited to your class, such as the probability of measuring an optimal or valid solution, the approximation ratio (class A), or the number of oracle calls and the success probability of the Grover search (classes B–D).

7.4. Deliverables

You should produce at least the following deliverables for Part II:

- The M2M transformations T1 and T2, and use the M2T generators for Qiskit Python.
- A fully working demonstration: from at least three problem specifications written in your DSL to generated code, execution and results in domain terms, reproducible from the repository with a single documented command.
- The validation results against the classical baseline (find a classical implementation to compare against).
- A small report (about 2 pages) explaining the transformations and their references, the validation results and the limitations of your approach.
- The LLM usage log for this part.

The deadline for these deliverables is November 11th.

8. Use of LLMs

You can use LLMs throughout the project: to explore the literature of your class of problems, to propose metamodel elements, to help with the transformation rules, and to explain Qiskit. However:

1. You are fully responsible for everything you submit. **Every group member is expected to be able to explain any part of the project in the final discussion.**

2. LLMs often get quantum formulations subtly wrong, for example: penalty weights that are too small, sign conventions in Ising mappings, qubit ordering (Qiskit is little-endian), or oracles that mark the wrong states. Your validation must catch these errors; an LLM is not an acceptable reference, only literature is.
3. Keep an **LLM usage log** in the repository: for each significant use, record the tool and model, the goal, a summary of the prompt, what was accepted, and **what was wrong and how you detected and fixed it**. The log is evaluated as a measure of your effort in critically reflecting and engaging with the project, not for its length.
4. The reports must be written by you. They should be **short and personal**, not generated.

9. Evaluation

Proposed weights, to be confirmed.

Component	Weight
Part I: domain analysis and concept lists	15%
Part I: metamodels and OCL (problem DSL, solution DSL)	20%
Part II: M2M transformations (T1, T2) and traceability to references	25%
Part II: M2T generator usage and working demonstration	10%
Part II: validation against classical baselines	10%
Reports and LLM usage log	10%

Originality of each group's view of the domain is explicitly valued. Individual marks are adjusted according to the contributions and the final discussion. An extra point may be given to the best project of each class (e.g. by using IBM's quantum computers for real execution).

[15/10/2026] Bonus: Mapping solutions back to the problem DSL will be a plus.

10. Final Remarks

Current quantum hardware is small and noisy, and for the instance sizes you can simulate, a classical computer will always be faster. That is expected. The point of the assignment is not quantum advantage but engineering: showing that MDE can turn a domain expert's problem statement into a correct quantum program through a traceable chain of transformations.

11. References

[1] M. Brambilla, J. Cabot, M. Wimmer. Model-Driven Software Engineering in Practice, 2nd ed. Morgan & Claypool, 2017. ISBN 978-1627057080.

[4] BESSER (Building bEtter Smart Software fastER): https://github.com/BESSER-PEARL/BESSER

[5] Eclipse Modeling Framework: https://eclipse.dev/modeling/emf/

[6] Qiskit documentation: https://quantum.cloud.ibm.com/docs

[8] F. Gemeinhardt, A. Garmendia, M. Wimmer. Towards model-driven quantum software engineering. Q-SE workshop at ICSE, 2021.

[9] M. A. Nielsen, I. L. Chuang. Quantum Computation and Quantum Information, 10th anniversary ed. Cambridge University Press, 2010.

[10] IBM Quantum Learning: https://quantum.cloud.ibm.com/learning

12. Annex A: Classes of Problems

This annex describes the four classes of problems of the assignment. They are built around **NP-complete problems and their NP-hard optimisation variants**, which is where quantum algorithms, such as Grover search and QAOA, are beneficial, as they enable a quadratic improvement over brute force approaches (but don't solve the problems in polynomial time). Each class has a pool of problems; each group is assigned one problem (domain) of one class and designs its own problem DSL for it. Problems marked * are assigned only if there are more groups than problems.

Test instances. Each group provides at least three instances of its problem: a tiny one checkable by hand, a medium one with a known solution, and one near the ~20-qubit limit. For the Grover classes (B, C, D), include also an instance with several solutions and, where possible, one with none.

Complexity labels. **NP-complete** refers to the decision version ("is there an x such that …?"); **NP-hard** to the optimisation version ("find the best x"), whose decision version ("is there an x with cost ≤ k?") is NP-complete. *Weakly* means that a pseudo-polynomial algorithm exists (dynamic programming in the size of the numbers), so instances with small numbers are easy classically. Complexity always refers to the general problem; the instances in the assignment are deliberately tiny, so a classical solver will always win on them (see Final Remarks). For weakly NP-complete problems (subset sum, partition, knapsack), state in your validation that dynamic programming solves such instances efficiently.

Overview

#	Question	Intermediate model (solution DSL)	Quantum algorithm	Complexity
A	Optimisation: minimise a quadratic cost under constraints (penalties)	QUBO / Ising: binary variables, linear and quadratic coefficients, offset; each penalty term traced to its constraint	QAOA (fixed schedule or optimiser loop)	**NP-hard**
B	Decision/search: logical constraints	Boolean formula (CNF) over binary-encoded variables	Grover: oracle synthesised from the formula, diffuser, iteration count	**NP-complete**
C	Decision/search: linear arithmetic constraints	Linear constraint system: integer variables with bit widths, linear expressions, =, ≤ (and, or); compiled by T2 into a reversible netlist	Grover with a compute–mark–uncompute oracle (ripple-carry or QFT adders, comparators)	**NP-complete**
D	Optimisation: linear objective under linear constraints (0/1 ILP)	Integer linear programme (ILP): class C model plus a linear objective	Grover Adaptive Search (Dürr–Høyer threshold search)	**NP-hard**

Classes A and D contain similar problem types (knapsack, independent set, assignment), which allows a direct comparison of QAOA (shallow, heuristic) with Grover Adaptive Search (deep, provable quadratic speedup over brute force). Classes B, C and D share the Grover machinery (oracle, diffuser, number of rounds).

Intermediate Models

Each group implements the intermediate (solution) model of its class as a metamodel (with OCL constraints) of its own design. The definitions below fix what an instance of that metamodel *means*: the mathematical object it denotes and the question asked about it. Solver choices (QAOA depth, penalty weights, number of Grover rounds) can be part of these objects, or they can be defined in a solver configuration model.

A. Optimisation with QAOA

Intermediate model: QUBO - Quadratic Unconstrained Binary Optimisation

Instance. *n* binary variables $x_1, x_2, \ldots, x_n$ (each 0 or 1) and real numbers: $a_1, \ldots, a_n$ (one per variable), $b_{12}, b_{13}, \ldots, b_{n-1,n}$ (one per pair of variables) and a constant *c*. It denotes the cost function:

$$f(x_1, \ldots, x_n) = c + a_1x_1 + a_2x_2 + \ldots + a_nx_n + b_{12}x_1x_2 + b_{13}x_1x_3 + \ldots + b_{n-1,n}x_{n-1}x_n$$

$$= c + \Sigma_i\, a_ix_i + \Sigma_{i<j}\, b_{ij}\, x_ix_n \quad \textit{(compact form)}$$

Question. Find the values of $x_1, \ldots, x_n$ with the lowest cost (optimisation).

Example: See slide 24, MaxCut problem.

Variants. If the original problem has hard constraints, they can be encoded as *penalty terms*. Assume the hard constraint g(**x**) = 0, with **x** = (x_1, x_2, …, x_n) and g(**x**) ≡ $d_0 + d_1x_1 + d_2x_2 + \ldots + d_nx_n$, with numbers $d_0, d_1, \ldots, d_n$. This is encoded by adding λ·g(**x**)² to $f(x_1, \ldots, x_n)$ with λ > 0 large enough that every violating choice costs more than any feasible one. Inequalities need *slack variables* first, e.g., g(**x**) ≤ 0 is encoded as g(**x**) + s = 0, where *s* is a slack variable to be minimised.

Problem pool

#	Problem (domain of the problem DSL)	Underlying problem	Complexity · notes
1	Network partitioning (split a network to maximise the weight of links between parts)	Weighted MaxCut	**NP-hard** · decision NP-complete (Karp, 1972)
2	Facility siting (place facilities without conflicts / cover all sites)	Maximum independent set / minimum vertex cover	**NP-hard** · decision NP-complete (Karp, 1972)
3	Frequency or exam-slot assignment	Graph colouring with k colours, minimising conflicts (one-hot encoding)	**NP-hard** · k-colourability NP-complete for k ≥ 3; for k = 2 it is in P, so use k ≥ 3
4	Portfolio or project selection under a budget	0/1 knapsack (slack variables for the capacity)	**NP-hard** · weakly; decision NP-complete
5	Shift or task assignment with capacities and incompatibilities	Generalised assignment problem	**NP-hard** · without capacities or conflicts it becomes the assignment problem, which is in P (Hungarian method)
6*	Diagnosis: most probable explanation of the evidence in a Bayesian network	MPE: minimise −Σ log $P(x_v \mid \text{parents})$; quadratic for single-parent nodes	**NP-hard** · decision NP-complete (Shimony, 1994)

Scope and references

Minimum scope. Your problem, with penalty weights computed automatically; QAOA with configurable depth *p* and a classical optimiser loop (or a fixed angle schedule); decoding of the measured bit strings into domain terms, with a feasibility check; at least three instances, up to ~20 qubits.

Stretch goals: constraint-preserving mixers instead of penalties (Hadfield et al.); higher-order terms (HUBO); comparison with Grover Adaptive Search (class D) on the same instance.

Starting references:

- A. Lucas, "Ising formulations of many NP problems", Frontiers in Physics, 2014.
- E. Farhi, J. Goldstone, S. Gutmann, "A Quantum Approximate Optimization Algorithm", arXiv:1411.4028, 2014.
- F. Glover, G. Kochenberger, Y. Du, "Quantum Bridge Analytics I: a tutorial on formulating and using QUBO models", 4OR, 2019.
- S. Hadfield et al., "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz", Algorithms, 2019.

B. Search with Logical Constraints (Grover)

Intermediate model: SAT - Satisfiability of Boolean formulas in CNF (conjunctive normal form)

Instance. *n* yes/no decision variables $x_1, x_2, \ldots, x_n$ (each 0 or 1) and *m* clauses $C_1, C_2, \ldots, C_m$. A *literal* is either a variable x_i (true when $x_i = 1$) or its negation $\neg x_i$ (true when $x_i = 0$). Each clause is a list of literals and is true when *at least one* of its literals is true. The formula is true when *all* clauses are true:

$$\varphi(x_1, \ldots, x_n) = C_1 \wedge C_2 \wedge \ldots \wedge C_m, \quad \text{where each } C_j = \ell_{j1} \vee \ell_{j2} \vee \ldots \text{ (literals)}$$

Question. Find values of $x_1, \ldots, x_n$ that make φ true, or report that there are none. There are $N = 2^n$ possible choices; if M of them make φ true, Grover needs about $(\pi/4)\cdot\sqrt{(N/M)}$ rounds.

Example: slide 18, party planning.

Variants. A decision variable with more than two values must be encoded in bits, and the bit patterns that do not stand for a value must be forbidden by extra clauses. E.g., a colour in {red, green, blue} as two bits x_1x_2 (00 = red, 01 = green, 10 = blue) needs the extra clause $(\neg x_1 \vee \neg x_2)$ to forbid 11.

Problem pool

#	Problem (domain of the problem DSL)	Underlying problem	Complexity · notes
1	Product configuration from a feature model (find a valid configuration with required/excluded features)	SAT (feature models translate to propositional formulas)	**NP-complete** · Cook, 1971; Batory, 2005
2	Sudoku (4×4, a few empty cells)	Sudoku completion	**NP-complete** · for $n^2 \times n^2$ grids (Yato & Seta, 2003)
3	N-queens completion / Latin square completion	Partial-placement completion	**NP-complete** · completion only (Gent et al., 2017; Colbourn, 1984); plain N-queens has a constructive solution

#	Problem (domain of the problem DSL)	Underlying problem	Complexity · notes
4	Map colouring (decision)	Graph 3-colourability	**NP-complete** · even for planar graphs; 4 colours always suffice for planar maps, so ask for 3
5	Logic puzzles (e.g. a small "Einstein" puzzle)	Finite-domain CSP / SAT	**NP-complete** · in general; each puzzle is a fixed instance
6*	Team formation (each skill covered exactly once)	Exact cover	**NP-complete** · Karp, 1972

Scope and references

Minimum scope. Encoding of the problem variables in bits, with clauses that forbid invalid bit patterns; oracle synthesised from the formula, with ancillas and uncomputation; diffuser; number of rounds computed from the number of solutions (taken from the classical baseline); up to ~12 search qubits plus ancillas.

Stretch goals: unknown number of solutions (Boyer et al.); fewer ancillas; counting solutions.

Starting references:

- L. Grover, "A fast quantum mechanical algorithm for database search", STOC, 1996.
- M. Boyer, G. Brassard, P. Høyer, A. Tapp, "Tight bounds on quantum searching", Fortschritte der Physik, 1998.
- Nielsen & Chuang [9], ch. 6.

C. Search with Linear Arithmetic Constraints (Grover + reversible arithmetic)

Intermediate model: Linear integer constraints

Instance. *n* integer unknowns $y_1, y_2, \ldots, y_n$. Each y_i has a number of bits w_i, so it takes the values 0, 1, …, 2^{w_i-1} (e.g. 3 bits: 0 to 7). The constraints are built from *atoms* of two forms (with a linear combination of the unknowns using integer coefficients on the left-hand side):

$$a_0 + a_1y_1 + a_2y_2 + \ldots + a_ny_n = 0 \quad \text{or} \quad a_0 + a_1y_1 + a_2y_2 + \ldots + a_ny_n \leq 0 \qquad a_0, a_1, \ldots, a_n \text{ integers}$$

combined with $\wedge$ and $\vee$.

Question. Find values of $y_1, \ldots, y_n$ that satisfy the constraints, or report that there are none.

Example (subset sum). Four items with weights 2, 3, 5, 7 and target 15: unknowns $y_1, \ldots, y_4$ with one bit each (1 = item chosen) and one atom:

$$2y_1 + 3y_2 + 5y_3 + 7y_4 - 15 = 0 \qquad \textit{only } y = (0, 1, 1, 1) \textit{ satisfies it}$$

Variants. Other comparisons are rewritten to the above two forms:

$$y_1 + y_2 \geq 3 \quad \text{becomes} \quad 3 - y_1 - y_2 \leq 0$$

$$y_1 < y_2 \quad \text{becomes} \quad y_1 - y_2 + 1 \leq 0$$

$$y_1 \neq y_2 \quad \text{becomes} \quad (y_1 - y_2 + 1 \leq 0) \vee (y_2 - y_1 + 1 \leq 0)$$

$$y_1 \leq 10 \text{ (a smaller domain)} \quad \text{becomes} \quad y_1 - 10 \leq 0$$

Quantum implementation notes. There are $N = 2^{w_1+\ldots+w_n}$ possible choices. Grover is used with an oracle that *computes* each expression into a register, *marks* the choices that satisfy the

constraints, and *uncomputes* the registers. Each expression $e = a_0 + a_1y_1 + \ldots + a_ny_n$ has a smallest and a largest possible value: for the smallest, take every term at its minimum ($y_i = 0$ if $a_i > 0$, $y_i = 2^{w_i} - 1$ if $a_i < 0$); for the largest, the opposite. A register with B bits in two's complement holds the values -2^{B-1} to $2^{B-1} - 1$, so B is the smallest number of bits that covers both values. In the example, the expression ranges from −15 (no item) to 2 (all items), so B = 5 bits (−16 to 15).

Problem pool

#	Problem (domain of the problem DSL)	Underlying problem	Complexity · notes
1	Exact payment / selection of items with a given total	Subset sum	**NP-complete** · weakly (Karp, 1972)
2	Knapsack feasibility (weight ≤ C and value ≥ V)	Decision knapsack	**NP-complete** · weakly
3	Change-making with bounded coin counts	Change-making	**NP-complete** · weakly (Lueker, 1975)
4	Load balancing of jobs on two machines with equal load	Partition	**NP-complete** · weakly (Karp, 1972)
5	Kakuro-style sum puzzles	Kakuro	**NP-complete** · Seta, 2002
6*	Packing items into k bins of given capacity	Bin packing (feasibility, fixed k)	**NP-complete** · weakly for fixed k

Scope and references

Minimum scope. Atoms with = and ≤ combined with and/or; register sizes derived from the model; reversible adders and comparators (ripple-carry or QFT-based) in a compute–mark–uncompute oracle; Grover search; each arithmetic block also tested exhaustively on basis inputs; up to ~20 qubits in total.

Stretch goals: both adder constructions compared (qubits, depth); products of variables (reversible multiplier); unknown number of solutions (Boyer et al.).

Starting references:

- V. Vedral, A. Barenco, A. Ekert, "Quantum networks for elementary arithmetic operations", Physical Review A, 1996.
- T. Draper, "Addition on a quantum computer", arXiv:quant-ph/0008033, 2000.
- S. Cuccaro et al., "A new quantum ripple-carry addition circuit", arXiv:quant-ph/0410184, 2004.
- L. Grover (1996) and M. Boyer et al. (1998), as for class B.

D. Optimisation with Grover Adaptive Search

Intermediate model: ILP - Integer Linear Programming

Instance. A class C instance (unknowns $y_1, \ldots, y_n$ with their bits, and constraints) plus a minimisation objective

$$g(y_1, \ldots, y_n) = c_0 + c_1y_1 + c_2y_2 + \ldots + c_ny_n \qquad c_0, c_1, \ldots, c_n \text{ integers}$$

Question. Among the choices that satisfy the constraints, find one with the smallest g (to maximise a value, minimise its negative).

Example (knapsack). Given a set of items with weights 2, 3, 5, 7 and values 3, 4, 8, 9, and a knapsack with capacity 10, select a subset of items with the maximum total value whose weights fit in the knapsack:

$$\text{constraint } 2y_1 + 3y_2 + 5y_3 + 7y_4 - 10 \leq 0, \quad \text{objective } g = -(3y_1 + 4y_2 + 8y_3 + 9y_4)$$

The optimum is y = (1, 1, 1, 0): weight 10, value 15.

Quantum implementation notes. The optimisation is turned into a sequence of class C searches with a threshold t (Grover Adaptive Search):

1. Start with any choice that satisfies the constraints (e.g. found by one class C search) and set t = g of that choice.
2. Search with Grover for a choice that satisfies the constraints and is strictly better than t, i.e. that also satisfies the atom g(y) − t + 1 ≤ 0. This is again a class C instance, so the class C oracle compiler is reused.
3. If one is found, set t to its g and go back to step 2.
4. Stop after a number of unsuccessful searches fixed in the solver configuration (Dürr–Høyer); the last choice found is the answer.

Problem pool

#	Problem (domain of the problem DSL)	Underlying problem	Complexity · notes
1	Best selection of items within a capacity	0/1 knapsack (optimisation)	**NP-hard** · weakly
2	Cheapest set of services covering all requirements	Minimum-cost set cover	**NP-hard** · Karp, 1972 (strongly)
3	Best set of non-conflicting activities (with weights)	Maximum-weight independent set (constraints $x_i + x_{\square} \leq 1$)	**NP-hard** · Karp, 1972
4	Cheapest assignment of tasks to agents under capacity	Generalised assignment problem	**NP-hard**
5	Budget allocation across projects with alternative options	Multiple-choice knapsack	**NP-hard** · weakly
6*	Most probable explanation in a Bayesian network	MPE with integer-scaled log-probabilities	**NP-hard** · needs products of bits, beyond the linear scope

Scope and references

Minimum scope. An oracle for the constraints plus the threshold atom g(y) − t + 1 ≤ 0, built as in class C; the adaptive loop of Grover Adaptive Search, with its parameters in the solver configuration; decoding of the best solution found; comparison with the exact optimum on at least three instances.

Stretch goals: comparison with QAOA (class A) on the same instance; randomised choice of the number of rounds (Boyer et al.).

Starting references:

– C. Dürr, P. Høyer, “A quantum algorithm for finding the minimum”, arXiv:quant-ph/9607014, 1996.
– A. Gilliam, S. Woerner, C. Gonciulea, “Grover Adaptive Search for Constrained Polynomial Binary Optimization”, Quantum 5, 428, 2021.

– T. Draper (2000) and S. Cuccaro et al. (2004), as for class C.

13. Annex B: A Small End-to-End Example

The following sketch illustrates the chain for class A on the smallest interesting instance: MaxCut on a triangle. The syntax shown is only an illustration; each group designs its own. It plays the same role as the GUI model example of last year's assignment.

1. Problem specification (problem DSL, concrete syntax)

The following example is presented in a concrete syntax, but the goal is for you to define a DSL to specify problems such as this, but comming from the class of problems assigned to you.

```
problem TriangleCut : MaxCut {
  graph { nodes a, b, c;  edges a--b, b--c, a--c; }
}
solver { algorithm QAOA; depth 1; shots 1024; }
```

2. Solution model (Ising form, following Lucas 2014)

One spin per node, z(a), z(b), z(c) ∈ {−1, +1}; minimise H = z(a)·z(b) + z(b)·z(c) + z(a)·z(c). Each edge contributes one quadratic term; an edge is cut when its two spins differ.

You need to define a DSL for representing solutions such as this one, but for your concrete class of problems. These model are generated from the models from the previous point (problem DSL) through T1.

3. Quantum model

The following code is a representation of the solution model using the quantum model available at BESSER. From the solution model, you should write the T2 transformation capable of generating code similar to this.

```
from besser.BUML.metamodel.quantum import (
    QuantumCircuit,
    QuantumRegister,
    ClassicalRegister,
    HadamardGate,
    CXGate,
    RZGate,
    RXGate,
    Measurement)

# ------------------------------------------------------------------------
# Fixed angle schedule (depth p=1) — these come from the solver config model,
# not derived here; placeholder values shown for a single QAOA layer.
# ------------------------------------------------------------------------
GAMMA_1 = math.pi / 8
BETA_1 = math.pi / 4

# ------------------------------------------------------------------------
# Registers: q0=a, q1=b, q2=c. No ancillas needed — Ising cost terms map
# directly onto ZZ interactions between the node qubits themselves.
# ------------------------------------------------------------------------
node_reg = QuantumRegister(size=3, name="nodes")
cls_reg = ClassicalRegister(size=3, name="result")

qa, qb, qc_ = node_reg.qubits[0], node_reg.qubits[1], node_reg.qubits[2]   # a, b, c

qc = QuantumCircuit(name="TriangleCut")
```

```
qc.add_qreg(node_reg)
qc.add_creg(cls_reg)

# ------------------------------------------------------------------------
# Initial state: uniform superposition |+>^3
# ------------------------------------------------------------------------
qc.add_operation(HadamardGate(qa))
qc.add_operation(HadamardGate(qb))
qc.add_operation(HadamardGate(qc_))

# ------------------------------------------------------------------------
# Cost layer U_C(gamma_1): one ZZ-rotation per edge: (a,b), (b,c), (a,c)
# ------------------------------------------------------------------------

# edge a--b
qc.add_operation(CXGate(control_qubit=qa, target_qubit=qb))
qc.add_operation(RZGate(theta=2 * GAMMA_1, target_qubit=qb))
#def __init__(self, target_qubit: int, theta: float, control_qubits: List[int] = None,
control_states: List[ControlState] = None):
qc.add_operation(CXGate(control_qubit=qa, target_qubit=qb))

# edge b--c
qc.add_operation(CXGate(control_qubit=qb, target_qubit=qc_))
qc.add_operation(RZGate(theta=2 * GAMMA_1, target_qubit=qc_))
qc.add_operation(CXGate(control_qubit=qb, target_qubit=qc_))

# edge a--c
qc.add_operation(CXGate(control_qubit=qa, target_qubit=qc_))
qc.add_operation(RZGate(theta=2 * GAMMA_1, target_qubit=qc_))
qc.add_operation(CXGate(control_qubit=qa, target_qubit=qc_))

# ------------------------------------------------------------------------
# Mixer layer U_B(beta_1): RX on every node qubit
# ------------------------------------------------------------------------
qc.add_operation(RXGate(theta=2 * BETA_1, target_qubit=qa))
qc.add_operation(RXGate(theta=2 * BETA_1, target_qubit=qb))
qc.add_operation(RXGate(theta=2 * BETA_1, target_qubit=qc_))

# ------------------------------------------------------------------------
# Measurement (1024 shots comes from the solver config, applied at execution,
# not part of the circuit model itself)
# ------------------------------------------------------------------------
qc.add_operation(Measurement([qa, qb, qc_], cls_reg))
```

4. Generated Qiskit code

The M2T generator then produces:

```
import sys
import os
sys.path = [p.strip() for p in sys.path if p]
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile

from qiskit_aer import AerSimulator

from qiskit.circuit.library import *
import numpy as np
```

```
# Initialize Registers

nodes = QuantumRegister(3, 'nodes')

result = ClassicalRegister(3, 'result')


# Initialize Circuit
qc = QuantumCircuit(nodes, result)

# Operations

qc.append(HGate(), [nodes[0]])

qc.append(HGate(), [nodes[1]])

qc.append(HGate(), [nodes[2]])

qc.append(CXGate(), [nodes[0], nodes[1]])

qc.append(RZGate(0.7853981633974483), [nodes[1]])

qc.append(CXGate(), [nodes[0], nodes[1]])

qc.append(CXGate(), [nodes[1], nodes[2]])

qc.append(RZGate(0.7853981633974483), [nodes[2]])

qc.append(CXGate(), [nodes[1], nodes[2]])

qc.append(CXGate(), [nodes[0], nodes[2]])

qc.append(RZGate(0.7853981633974483), [nodes[2]])

qc.append(CXGate(), [nodes[0], nodes[2]])

qc.append(RXGate(1.5707963267948966), [nodes[0]])

qc.append(RXGate(1.5707963267948966), [nodes[1]])

qc.append(RXGate(1.5707963267948966), [nodes[2]])

# Draw
print(qc.draw())

qc.measure(nodes, result)

# Execution (Local Aer Simulator)
print("Simulating circuit with Aer Simulator...")
simulator = AerSimulator()
transpiled_qc = transpile(qc, simulator)
sim_result = simulator.run(transpiled_qc, shots=1024).result()
counts = sim_result.get_counts()
print("Counts:", counts)
```